

HTML , CSS , JS 개념과 역할

1. HTML (HyperText Markup Language)

- 비유: 집의 '뼈대와 구조 (골조)'

개념

HTML은 웹페이지의 **구조와 텍스트 내용**을 정의하는 마크업 언어입니다. 화면에 제목, 문단, 이미지, 버튼, 링크 등을 어떤 순서와 관계로 배치할지 웹브라우저에 알려주는 역할을 합니다.

주요 역할

- **콘텐츠 정의:** 텍스트, 이미지, 동영상 등 웹사이트에 들어갈 모든 요소를 배치합니다.
- **구조화:** 문서의 머리말(`<header>`), 본문(`<main>`), 바닥글(`<footer>`) 등 영역을 나누어 의미를 부여합니다.
- **링크 연결:** 하이퍼텍스트를 통해 다른 페이지나 사이트로 이동할 수 있는 링크를 만듭니다.

2. CSS (Cascading Style Sheets)

- 비유: 집의 '인테리어와 외관 (색상, 가구 배치, 스타일)'

개념

CSS는 HTML로 만들어진 뼈대에 **옷을 입히고 꾸며주는** 스타일 시트 언어입니다. 글꼴, 색상, 크기, 레이아웃, 배경 등 웹페이지의 시각적인 디자인을 담당합니다.

주요 역할

- **시각적 디자인:** 글자 색상, 크기, 폰트, 배경 이미지 등을 보기 좋게 꾸밈니다.
- **레이아웃 배치:** 요소들의 위치, 간격, 정렬 방식을 지정합니다. (예: Flexbox, Grid 등 사용)
- **반응형 웹 구현:** 스마트폰, 태블릿, PC 등 다양한 기기 화면 크기에 맞춰 디자인이 유연하게 변하도록(Media Query) 만듭니다.

3. JavaScript (JS)

- 비유: 집의 '전기 시스템, 문 자동 개폐 장치 등 동적인 기능'

개념

JavaScript는 웹페이지에 **생동감을 불어넣는** 프로그래밍 언어입니다. 단순히 멈춰 있는 화면이 아니라, 사용자의 행동에 반응하여 움직이거나 데이터를 처리하는 **동적 기능**을 구현합니다.

주요 역할

- **사용자 상호작용:** 버튼 클릭, 마우스 오버, 입력 폼 제출 등의 행동에 반응합니다. (예: 팝업창 띄우기, 메뉴 펼치기)
- **동적 콘텐츠 제어:** 페이지를 새로고침하지 않고도 실시간으로 내용을 변경합니다. (예: 무한 스크롤, 좋아요 버튼 카운트 증가)
- **데이터 처리:** 서버와 통신(API)하여 필요한 데이터를 가져와 화면에 보여줍니다. (예: 날씨 정보 업데이트, 실시간 검색어)

💡 **요약 비교 (건축 비유)** 기술역할비유 (건축) HTML 뼈대와 구조를 잡음 콘크리트 벽과 방 구조 CSS 디자인과 스타일을 꾸밈 페인트칠, 벽지, 가구 배치 JS 움직임과 기능을 부여함 자동문, 전등 스위치, 보일러 조작

웹페이지는 **HTML(구조)로 뼈대를 세우고, CSS(스타일)로 예쁘게 색칠한 뒤, JavaScript(기능)로 움직이게 만드는** 3가지 요소의 협업으로 완성됩니다.

구분	HTML	CSS	JavaScript (JS)
정의	HyperText Markup Language	Cascading Style Sheets	프로그래밍 언어 (스크립트 언어)
웹에서의 역할	구조(Structure) 및 콘텐츠 정의	스타일(Style) 및 레이아웃 꾸미기	동작(Behavior) 및 상호작용 제어
건축 비유	콘크리트 벽과 방 구조 (뼈대)	벽지, 페인트칠, 가구 배치 (인테리어)	자동문, 전등 스위치, 보일러 (전기/기계 장치)
핵심 요소	태그 (<code><div></code> , <code><h1></code> 등) 시맨틱 웹 (<code>header</code> , <code>footer</code>)	박스 모델 (<code>margin</code> , <code>padding</code>) 레이아웃 (<code>Flexbox</code> , <code>Grid</code>)	DOM 조작 이벤트 리스너 (클릭 등) 데이터 처리 및 API 연동
주요 특징	웹페이지의 뼈대를 세우고 의미를 부여함	시각적 요소를 제어하며 반응형 웹을 구현함	사용자의 행동에 반응하고 실시간으로 화면을 변경함
코드 예시	<pre><button class="btn">클릭</button></pre>	<pre>.btn { background-color: blue; }</pre>	<pre>btn.addEventListener('click', ...)</pre>

HTML 개념 정리

1. HTML이란 무엇인가?

- **정의:** 웹페이지의 구조와 의미(Content & Structure)를 정의하는 마크업 언어입니다.
- **역할:** 화면에 글자, 이미지, 동영상, 버튼 같은 요소들이 어떤 형태로 배치될지 웹 브라우저(Chrome, Safari 등)에 알려줍니다. 프로그래밍 언어가 아니라, 정보를 구조화하는 마크업 언어입니다.

2. HTML 문서의 기본 구조

모든 HTML 문서는 일정한 뼈대(템플릿)를 가지고 있습니다.

HTML

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>웹페이지 제목</title>
</head>
<body>
  <h1>안녕하세요!</h1>
  <p>웹페이지 본문 내용입니다.</p>
</body>
</html>
```

- **<!DOCTYPE html>**: 이 문서가 최신 웹 표준인 **HTML5**로 작성되었음을 브라우저에 알립니다.
- **<html>**: HTML 문서의 최상위 루트(Root) 요소로, 모든 코드가 이 안에 들어갑니다.
- **<head>**: 웹페이지의 보이지 않는 정보(메타데이터)를 담는 곳입니다. 검색엔진 최적화(SEO), 문자 인코딩 방식(**UTF-8**), 웹페이지 제목(**<title>**), CSS 파일 연결 등을 처리합니다.
- **<body>**: 웹페이지의 실제 화면에 보이는 모든 콘텐츠(텍스트, 이미지, 버튼 등)가 들어가는 공간입니다.

3. HTML의 기본 문법 (태그와 속성)

HTML은 태그(Tag)라는 기호를 사용하여 요소를 감싸는 방식으로 작성됩니다.

- **태그 구조:** **<태그이름>콘텐츠</태그이름>** (대부분 열고 닫는 쌍으로 이루어짐)

- **빈 태그 (Self-Closing Tag):** 내용이 없고 스스로 닫히는 태그도 있습니다. (예: `<br>`, `<img>`, `<hr>`)
- **속성 (Attribute):** 태그에 추가적인 정보를 제공하는 설정 값입니다.HTML

```
<a href="https://www.google.com" target="_blank">구글로 이동</a>
```

- `href` 와 `target` 은 속성(Attribute)이며, 링크의 목적지나 열리는 방식을 지정합니다.

4. 핵심 HTML 요소 (종류별 정리)

① 텍스트 및 구조 관련 태그

- `<h1>` ~ `<h6>`: 제목을 나타냅니다. 숫자가 작을수록(h1) 가장 중요한 큰 제목이며, SEO(검색엔진 최적화)에 큰 영향을 미칩니다.
- `<p>`: 문단(Paragraph)을 나눕니다.
- `<br>`: 줄 바꿈을 합니다.
- `<hr>`: 수평 줄을 그어 화면을 분리합니다.

② 목록(List) 태그

- `<ul>` (Unordered List): 순서가 없는 목록 (점 모양 불릿)
- `<ol>` (Ordered List): 순서가 있는 목록 (숫자나 알파벳)
- `<li>` (List Item): `<ul>` 이나 `<ol>` 내부에서 각 항목을 정의할 때 사용합니다.

③ 미디어 및 링크 태그

- `<a>` (Anchor): 다른 페이지나 사이트로 이동하는 하이퍼링크를 만듭니다 (`href` 속성 필수).
- `<img>`: 이미지를 삽입합니다 (`src` 속성에 이미지 경로 지정).
- `<video>`, `<audio>`: 동영상과 오디오 파일을 재생합니다.

④ 폼(Form) 및 입력 태그 (사용자 상호작용)

- `<form>`: 사용자가 입력한 데이터를 서버로 전송하는 감싸기 박스입니다.
- `<input>`: 텍스트, 비밀번호, 체크박스 등 사용자로부터 입력을 받는 칸을 만듭니다 (`type="text"`, `type="password"` 등).

- `<button>`: 클릭 가능한 버튼을 만듭니다.

5. 가장 중요한 개념: 시맨틱 웹 (Semantic Web)

HTML5 이후로 가장 강조되는 개념으로, "의미가 있는 태그를 올바르게 사용하자"는 뜻입니다. 단순히 `<div>` (의미 없는 박스)만 남발하는 것이 아니라, 각 구역의 역할에 맞는 태그를 사용합니다.

- `<header>`: 상단 머리말 영역 (로고, 사이트 제목 등)
- `<nav>`: 네비게이션 메뉴 영역 (다른 페이지로 이동하는 링크 모음)
- `<main>`: 문서의 핵심 콘텐츠가 들어가는 메인 영역
- `<section>`: 주제별 콘텐츠 묶음 영역
- `<article>`: 독립적으로 소비 가능한 콘텐츠 (뉴스 기사, 블로그 포스트 등)
- `<footer>`: 하단 바닥글 영역 (저작권, 연락처 정보 등)

시맨틱 태그를 쓰는 이유

1. **검색엔진 최적화(SEO)**: 검색엔진 봇이 웹사이트의 구조를 정확히 파악하여 검색 결과 상위에 노출될 확률을 높여줍니다.
2. **웹 접근성(Accessibility)**: 시각장애인이 사용하는 스크린 리더(Screen Reader)가 웹페이지의 구조를 정확히 읽어낼 수 있습니다.
3. **코드 가독성**: 다른 개발자가 코드를 보았을 때 유지보수하기가 훨씬 수월합니다.

CSS 개념 정리

1. CSS란 무엇인가?

- **정의:** HTML로 작성된 뼈대에 스타일(색상, 크기, 폰트)과 레이아웃(배치)을 입히는 스타일 시트 언어입니다.
- **핵심 철학 (Separation of Concerns):** 콘텐츠(HTML)와 디자인(CSS)을 완전히 분리하여 관리함으로써, 웹사이트의 유지보수를 획기적으로 쉽게 만들어 줍니다.

CSS를 HTML에 적용하는 3가지 방법

1. **외부 스타일 시트 (External CSS):** `.css` 파일을 따로 만들어 `<link>` 태그로 연결합니다. (가장 권장되는 방식)
2. **내부 스타일 시트 (Internal CSS):** HTML 문서 내부 `<head>` 태그 안에 `<style>` 태그를 열어 작성합니다.
3. **인라인 스타일 (Inline CSS):** HTML 태그 안에 `style="..."` 속성을 직접 작성합니다. (유지보수가 어려워 특수한 경우가 아니면 지양합니다.)

HTML

```
<!-- 외부 스타일 시트 연결 예시 -->
<link rel="stylesheet" href="style.css">
```

2. CSS 기본 문법과 선택자 (Selectors)

CSS는 어떤 요소(Selector)를 선택해서, 어떤 속성(Property)에 어떤 값(Value)을 줄 것인지 지정하는 구조로 이루어져 있습니다.

CSS

```
/* 기본 문법 구조 */
선택자 {
  속성: 값;
  속성: 값;
}

/* 실제 예시 */
h1 {
  color: blue;
```

```
font-size: 24px;
}
```

주요 선택자의 종류

- **전체 선택자 ()**: 페이지의 모든 요소를 선택합니다.
- **요소(태그) 선택자 (`p`, `h1` 등)**: 특정 태그를 가진 모든 요소를 선택합니다.
- **클래스 선택자 (`.class-name`)**: 특정 클래스가 지정된 요소를 선택합니다. (여러 개 사용 가능)
- **아이디 선택자 (`#id-name`)**: 특정 ID가 지정된 유일한 요소를 선택합니다. (페이지 내 단 1개만 존재해야 함)
- **자손 및 자식 선택자 (`div p` vs `div > p`)**:
 - `div p`: div 안에 있는 모든 p 태그 (자손)
 - `div > p`: div 바로 아래에 있는 p 태그 (직계 자식)
- **가상 클래스 (Pseudo-Classes)**: 요소의 특정 상태를 선택합니다. (예: 마우스를 올렸을 때 `:hover`, 활성화되었을 때 `:active`)

3. 가장 중요한 개념: CSS 박스 모델 (Box Model)

CSS의 모든 HTML 요소는 눈에 보이든 안 보이든 **사각형 박스**로 둘러싸여 있습니다. 이 박스의 구조를 이해하는 것이 레이아웃 배치의 핵심입니다.

안쪽부터 바깥쪽 순서로 4가지 영역으로 나뉩니다.

1. **Content (콘텐츠)**: 텍스트나 이미지 등 실제 내용이 들어가는 공간 (`width`, `height` 로 제어)
2. **Padding (안쪽 여백)**: 콘텐츠와 테두리(Border) 사이의 여백
3. **Border (테두리)**: 콘텐츠와 패딩을 둘러싸고 있는 선
4. **Margin (바깥 여백)**: 테두리와 다른 요소 사이의 간격

💡 **필수 설정**: `box-sizing: border-box`

기본적으로 브라우저는 요소의 너비(`width`)를 계산할 때 패딩과 테두리를 따로 더해서 계산합니다. 이 때문에 레이아웃이 깨지는 현상이 자주 발생합니다. 실무에서는 모든 요소에 아래 설정을 필수로 적용하여 패딩과 테두리가 너비 안에 포함되도록 만듭니다.

```
* {
  box-sizing: border-box;
}
```

```
}
```

4. 현대의 레이아웃 시스템 (Flexbox와 Grid)

요소를 가로로 정렬하거나 화면 배치를 바꿀 때 사용하는 핵심 레이아웃 도구입니다.

① Flexbox (Flexible Box)

- **특징: 한 차원(1차원)** 즉, 행(Row) 또는 열(Column) 중 한 방향으로 요소를 정렬할 때 최적화되어 있습니다.
- **주요 속성:**
 - `display: flex;` (부모 요소에 지정)
 - `justify-content: space-between;` (가로 방향 정렬)
 - `align-items: center;` (세로 방향 정렬)

② CSS Grid

- **특징: 두 차원(2차원)** 즉, 가로와 세로를 동시에 격자(Grid) 형태로 나누어 복잡한 레이아웃을 짤 때 사용합니다.
- **주요 속성:**
 - `display: grid;`
 - `grid-template-columns: repeat(3, 1fr);` (3개의 동일한 너비를 가진 열 생성)

5. CSS의 핵심 원리: 캐스케이딩과 우선순위 (Specificity)

CSS라는 이름에 들어간 **Cascading**은 "폭포처럼 아래로 흐른다"는 뜻으로, 스타일이 충돌할 때 어떤 스타일을 적용할지 결정하는 규칙을 의미합니다.

스타일이 충돌할 때 적용되는 우선순위 규칙

1. **중요도 (!important):** 무조건 최우선으로 적용됩니다. (유지보수를 위해 남발하면 안 됩니다.)
2. **명시도 (Specificity):** 얼마나 구체적으로 지정했느냐에 따라 결정됩니다.
 - 인라인 스타일 (1000점) > ID 선택자 (`#id`, 100점) > 클래스/가상 클래스 선택자 (`.class`, 10점) > 요소 선택자 (`div`, 1점)
3. **소스 코드의 순서:** 명시도가 같다면, 나중에 작성된(아래에 있는) 코드가 이깁니다.

6. 반응형 웹과 최신 CSS 기능

① 미디어 쿼리 (Media Queries)

스마트폰, 태블릿, PC 등 기기의 화면 크기에 따라 디자인을 동적으로 변경합니다.

CSS

```
/* 화면 너비가 768px 이하일 때 적용되는 스타일 (모바일 환경) */
@media (max-width: 768px) {
  body {
    font-size: 14px;
    background-color: #f0f0f0;
  }
}
```

② CSS 변수 (Custom Properties)

사이트 전체에서 공통으로 쓰는 색상이나 폰트를 변수로 선언해 재사용할 수 있습니다.

CSS

```
:root {
  --main-color: #007bff;
  --text-color: #333333;
}

button {
  background-color: var(--main-color);
  color: var(--text-color);
}
```

Java Script 개념 정리

1. JavaScript란 무엇인가?

- **정의:** 웹페이지에 생동감을 부여하고 사용자의 행동에 반응하게 만드는 프로그래밍 언어입니다.
- **역할:** HTML이 집의 뼈대, CSS가 인테리어라면, 자바스크립트는 **전기 시스템, 자동문, 보일러 조작 장치**처럼 '움직이고 기능을 수행하는 시스템' 역할을 합니다.
- **특징:**
 - 브라우저에서 직접 실행되는 인터프리터 언어입니다.
 - 웹뿐만 아니라 Node.js를 통해 서버(Backend) 개발, 모바일 앱, 데스크톱 앱까지 개발할 수 있는 범용 언어로 진화했습니다.

2. JavaScript의 기본 문법과 데이터 다루기

변수와 데이터 타입

변수는 데이터를 담는 상자입니다. 최신 자바스크립트에서는 `let` (값이 바뀔 수 있는 변수) 과 `const` (상수, 값이 바뀌지 않음)를 사용합니다.

JavaScript

```
// 변수 선언과 데이터 저장
let userName = "홍길동"; // 문자열 (String)
const age = 25;           // 숫자 (Number)
let isLogin = true;       // 불리언 (Boolean, 참/거짓)

// 배열 (Array)과 객체 (Object)
let hobbies = ["코딩", "독서", "운동"];
let user = {
  name: "김철수",
  job: "개발자",
  greet: function(){
    console.log("안녕하세요!");
  }
};
```

조건문과 반복문

JavaScript

```
// 조건문 (if-else)
let score = 85;

if (score >= 90) {
    console.log("A 학점입니다.");
} else if (score >= 80) {
    console.log("B 학점입니다.");
} else {
    console.log("C 학점입니다.");
}

// 반복문 (for)
for (let i = 0; i < 3; i++) {
    console.log(`현재 숫자는 ${i}입니다.`);
}
```

함수 (Functions)

코드를 묶어두고 필요할 때마다 재사용하는 기능입니다. 최신 문법인 화살표 함수(Arrow Function)를 주로 사용합니다.

JavaScript

```
// 화살표 함수 정의
const calculateSum = (a, b) => {
    return a + b;
};

let result = calculateSum(10, 20); // 결과: 30
```

3. 웹 제어의 핵심: DOM (Document Object Model)

자바스크립트가 HTML과 CSS를 조작할 수 있는 이유는 **DOM** 덕분입니다. 브라우저는 HTML 문서를 메모리 위에 **객체(Object) 트리 구조**로 변환하는데, 이를 DOM이라고 부릅니다.

자바스크립트는 이 DOM을 통해 요소를 선택하고, 내용을 바꾸거나 스타일을 수정할 수 있습니다.

JavaScript

```
// 1. HTML 요소를 선택하기
const titleElement = document.querySelector('h1');
const buttonElement = document.querySelector('#submit-btn');

// 2. 요소의 텍스트나 스타일 변경하기
```

```
titleElement.textContent = "자바스크립트로 변경된 제목!";
titleElement.style.color = "red";
```

4. 사용자 상호작용: 이벤트 (Events)

사용자가 웹페이지에서 하는 모든 행동(클릭, 키보드 입력, 스크롤, 마우스 올리기 등)을 이벤트(Event)라고 합니다. 자바스크립트는 이벤트를 감지하여 특정 코드를 실행할 수 있습니다.

JavaScript

```
const button = document.querySelector('.my-button');

// 이벤트 리스너: 사용자가 'click' 행동을 할 때까지 기다림
button.addEventListener('click', function(){
    alert('버튼이 클릭되었습니다!');

    // 클릭 시 배경색을 노란색으로 변경
    document.body.style.backgroundColor = "yellow";
});
```

5. 비동기 처리와 서버 통신 (Asynchronous JS)

웹페이지를 새로고침하지 않고도 서버에서 데이터를 가져와 화면을 실시간으로 업데이트하는 기술입니다. 이를 위해 비동기 처리(Asynchronous)가 필수적입니다.

- **AJAX / Fetch API:** 서버에 데이터를 요청(Request)하고 응답(Response)을 받는 기능입니다.
- **Promise와 async/await:** 비동기 작업이 끝날 때까지 기다렸다가 다음 코드를 실행하게 해주는 현대 자바스크립트의 핵심 문법입니다.

JavaScript

```
// 서버에서 날씨 정보를 비동기로 가져오는 예시 코드
async function getWeather(){
    try {
        let response = await fetch('https://api.example.com/weather');
        let data = await response.json();

        console.log("받아온 날씨 데이터:", data);
    } catch (error) {
        console.error("데이터를 불러오는 중 에러가 발생했습니다:", error);
    }
}
```

```
getWeather();
```

6. JavaScript의 동작 원리 (핵심 개념)

자바스크립트가 어떻게 코드를 실행하고 브라우저에서 작동하는지 이해하는 핵심 키워드입니다.

1. **싱글 스레드 (Single Thread):** 자바스크립트는 기본적으로 한 번에 하나의 작업만 처리할 수 있는 싱글 스레드 언어입니다.
2. **이벤트 루프 (Event Loop):** 싱글 스레드의 한계를 극복하기 위해, 브라우저는 백그라운드에서 비동기 작업(타이머, 서버 통신 등)을 처리한 뒤 작업이 끝나면 콜백 함수를 태스크 큐(Task Queue)에 넣고 이벤트 루프가 이를 순서대로 호출(Call Stack)하여 실행합니다.
3. **클로저 (Closure):** 함수가 생성될 당시의 주변 환경(상태)을 기억하고, 함수가 외부로 리턴된 이후에도 그 환경에 접근할 수 있는 강력한 기능입니다.

Flex 와 Grid 의 차이점

1. Flexbox vs Grid 요약 비교 표

구분	Flexbox (Flexible Box)	CSS Grid (Grid Layout)
차원 (Dimension)	1차원 (1-Dimensional) (행 또는 열 중 한 방향으로만 제어)	2차원 (2-Dimensional) (가로와 세로를 동시에 행과 열로 제어)
접근 방식	콘텐츠 중심 (Content-out) 내용물의 크기에 맞춰 공간을 배분함	레이아웃 중심 (Layout-in) 전체 판을 먼저 짜고 그 안에 요소를 배치함
주요 사용 사례	네비게이션바, 버튼 그룹, 카드 내부 정렬 등 일렬 배치	웹사이트 전체 레이아웃(헤더, 사이드바, 본문, 푸터 등)
핵심 속성 (부모)	<code>display: flex;</code> <code>justify-content</code> , <code>align-items</code>	<code>display: grid;</code> <code>grid-template-columns</code> , <code>grid-template-rows</code>

2. Flexbox (플렉스박스): 1차원 레이아웃

개념과 특징

- Flexbox는 한 번에 한 방향(가로 혹은 세로)으로만 요소를 배치할 때 사용하는 1차원 레이아웃 시스템입니다.
- 부모 요소에 `display: flex;` 를 선언하는 순간, 내부의 자식 요소들은 주축(Main Axis)을 따라 나란히 정렬됩니다.
- 콘텐츠의 크기에 유연하게 반응하므로, 컴포넌트 내부의 정렬(예: 버튼 옆에 아이콘 배치 등)에 매우 뛰어납니다.

주요 방향성 (Axis)

- `flex-direction: row;` (기본값): 가로 방향으로 정렬 (행 기준)
- `flex-direction: column;` 조합: 세로 방향으로 정렬 (열 기준)

코드 예시 (가로 정렬 및 간격 균등 배치)

CSS

```
.container {
  display: flex;
  flex-direction: row; /* 가로 방향 정렬 */
  justify-content: space-between; /* 양 끝 정렬 및 여백 균등 배분 */
  align-items: center; /* 세로 가운데 정렬 */
}
```

3. CSS Grid (그리드): 2차원 레이아웃

개념과 특징

- Grid는 **가로(행, Row)**와 **세로(열, Column)**를 동시에 격자(Grid) 형태로 나누어 관리하는 2차원 레이아웃 시스템입니다.
- 전체 웹페이지의 큰 틀(뼈대)을 짤 때 압도적인 위력을 발휘합니다. 격자판을 먼저 그려 놓고 원하는 위치에 요소를 적절 배치하는 방식입니다.

주요 개념

- **Track:** 행(Row)과 열(Column)의 선과 선 사이의 공간
- **Cell:** 가장 작은 단위의 격자 칸
- **Area:** 여러 개의 셀을 합친 영역 (예: 사이드바 영역, 본문 영역 등)

코드 예시 (3단 레이아웃 그리드 생성)

CSS

```
.container {
  display: grid;
  /* 3개의 동일한 너비를 가진 열(Column) 생성 */
  grid-template-columns: repeat(3, 1fr);
  /* 각 행의 높이 지정 */
  grid-template-rows: 100px 300px 100px;
  gap: 20px; /* 격자 사이의 간격 */
}
```

4. 둘을 언제 어떻게 선택해야 할까?

- **Flexbox를 써야 할 때:**
 - 요소를 **한 줄**로 쭉 나열해야 할 때 (예: 메뉴 바, 버튼 리스트)
 - 요소들의 크기가 유동적이고, 내부 콘텐츠에 따라 크기가 결정되어야 할 때

- 정렬 방향이 한 방향(가로 혹은 세로)으로만 움직일 때
- **Grid를 써야 할 때:**
 - 화면을 **가로와 세로 양쪽**으로 동시에 쪼개서 배치해야 할 때 (예: 대시보드 화면, 매거진 스타일의 이미지 갤러리)
 - 전체 웹사이트의 뼈대(Header, Sidebar, Main Content, Footer) 구조를 잡을 때